**KONKURS PROGRAMISTYCZNY „SCHOOL MUSIC”**

**1. Temat konkursu**

**„School Music – muzyka, którą wybiera szkoła”**

Zadaniem uczestników jest zaprojektowanie i wykonanie nowoczesnego systemu informatycznego umożliwiającego zarządzanie muzyką odtwarzaną podczas przerw szkolnych.

System ma łączyć **aplikację serwerową, aplikację webową, aplikację mobilną oraz bazę danych**, a jego główną funkcją jest umożliwienie uczniom głosowania na utwory muzyczne, które następnie są umieszczane w kolejce odtwarzania podczas szkolnych przerw.

Projekt powinien zostać wykonany w taki sposób, aby możliwe było jego rzeczywiste wykorzystanie w szkole.

**2. Cel konkursu**

Celem konkursu jest stworzenie kompletnego rozwiązania informatycznego, które pozwoli:

- zarządzać bazą utworów muzycznych,
- planować przerwy lekcyjne,
- określać, podczas których przerw odtwarzana jest muzyka,
- umożliwić uczniom głosowanie na utwory,
- tworzyć ranking najpopularniejszych utworów,
- automatycznie tworzyć kolejkę odtwarzania,
- kontrolować czas trwania przerwy,
- prezentować aktualnie odtwarzany utwór,
- gromadzić historię głosowań i odtworzeń.

Projekt powinien być nie tylko demonstracją umiejętności programistycznych, ale również rozwiązaniem odpowiadającym na rzeczywistą potrzebę szkoły.

**3. Zadanie konkursowe**

Uczestnicy mają stworzyć system **School Music**, w którym centralnym elementem jest serwer aplikacji oraz baza danych.

System powinien składać się co najmniej z:

1. **aplikacji serwerowej**,
2. **bazy danych**,
3. **REST API**,
4. **panelu administracyjnego WWW**,
5. **aplikacji mobilnej dla uczniów**.

System powinien obsługiwać skalowalną listę minimum **100 utworów muzycznych**.

Uczniowie korzystający z aplikacji mobilnej mogą oddawać głosy na wybrane utwory. Na podstawie zgromadzonych głosów system tworzy ranking, a następnie kolejkę utworów przeznaczonych do odtworzenia podczas najbliższej przerwy.

**4. Harmonogram przerw**

System musi posiadać moduł zarządzania harmonogramem przerw.

Administrator powinien mieć możliwość zdefiniowania np.:

| Przerwa | Początek | Koniec | Muzyka |
|---|---|---|---|
| 1 | 08:45 | 08:55 | TAK |
| 2 | 09:40 | 09:50 | TAK |
| 3 | 10:35 | 10:45 | TAK |
| 4 | 11:30 | 11:40 | NIE |
| 5 | 12:25 | 12:45 | TAK |

Godziny są przykładowe i powinny być możliwe do zmiany przez administratora.

System powinien automatycznie określać:

- czy aktualnie trwa lekcja,
- czy trwa przerwa,
- kiedy rozpocznie się kolejna przerwa,
- ile czasu pozostało do rozpoczęcia przerwy,
- ile czasu pozostało do jej zakończenia,
- czy podczas danej przerwy ma być odtwarzana muzyka.

**Przykład**

Jeżeli jest godzina 10:27, aplikacja może wyświetlać:

**NAJBLIŻSZA PRZERWA**

10:35–10:45

**Do rozpoczęcia: 08:00**

**Muzyka: TAK**

Po rozpoczęciu przerwy licznik powinien zmienić się na:

**PRZERWA TRWA**

**Pozostało: 14:32**

**5. Kalendarz**

System powinien posiadać kalendarz umożliwiający zarządzanie harmonogramem w dłuższym okresie.

Administrator powinien mieć możliwość:

- ustawienia harmonogramu dla całego tygodnia,
- ustawienia harmonogramu dla konkretnych dni,
- wyłączenia muzyki podczas wybranej przerwy,
- wyłączenia muzyki w całym dniu,
- ustawienia dni wolnych,
- utworzenia wyjątków od standardowego harmonogramu,
- przeglądania harmonogramu w widoku kalendarza.

Przykład:

**Poniedziałek** – muzyka podczas wszystkich przerw
**Wtorek** – muzyka podczas przerw 1–4
**Środa** – dzień bez muzyki
**Czwartek** – muzyka podczas wszystkich przerw
**Piątek** – muzyka podczas wybranych przerw

**6. Baza utworów**

System powinien posiadać bazę minimum **100 utworów**.

Każdy utwór powinien zawierać co najmniej:

- tytuł,
- wykonawcę,
- link do utworu,
- źródło utworu,
- czas trwania,
- grafikę/okładkę,
- gatunek lub kategorię,
- liczbę głosów.

Administrator powinien mieć możliwość dodawania, edytowania oraz usuwania utworów.

Utwory mogą pochodzić np. z:

- Spotify,
- YouTube,
- innych legalnych źródeł muzyki.

Integracja z zewnętrznym serwisem powinna odbywać się za pomocą dostępnego API lub zgodnego z regulaminem mechanizmu.

**7. Głosowanie**

Aplikacja mobilna powinna umożliwiać uczniom głosowanie na utwory.

Uczeń powinien móc:

- przeglądać listę utworów,
- wyszukiwać utwory,
- zobaczyć wykonawcę i okładkę,
- sprawdzić liczbę głosów,
- oddać głos,
- wycofać głos, jeśli system przewiduje taką funkcję,

- zobaczyć aktualny ranking.

System powinien zabezpieczać głosowanie przed wielokrotnym oddawaniem głosu na ten sam utwór przez tego samego użytkownika.

Przykładowy ranking:

**1. Imagine Dragons – Believer – 84 głosy**

**2. The Weeknd – Blinding Lights – 72 głosy**

**3. David Guetta – Titanium – 65 głosów**

**8. Kolejka odtwarzania**

Na podstawie wyników głosowania system powinien utworzyć kolejkę utworów.

Przykład:

**Przerwa 10:35–10:50**

Pozostały czas: **15 minut**

Kolejka:

1. Utwór A – 3:42
2. Utwór B – 3:15
3. Utwór C – 4:01
4. Utwór D – 3:37

System powinien uwzględniać **czas trwania przerwy** i dobierać utwory tak, aby możliwie efektywnie wykorzystać dostępny czas.

Po zakończeniu przerwy system powinien przygotować kolejkę dla kolejnej przerwy.

**9. Aplikacja mobilna**

Aplikacja mobilna jest przeznaczona przede wszystkim dla uczniów.

Powinna umożliwiać:

- logowanie,
- przeglądanie utworów,
- wyszukiwanie utworów,
- głosowanie,

- podgląd rankingu,
- sprawdzenie najbliższej przerwy,
- sprawdzenie czasu do rozpoczęcia przerwy,
- sprawdzenie czasu pozostałego do zakończenia przerwy,
- sprawdzenie aktualnie odtwarzanego utworu,
- podgląd kolejki.

**Przykładowy ekran**

**SCHOOL MUSIC**

🎵 Aktualnie:

**The Weeknd – Blinding Lights**

**NAJBLIŻSZA PRZERWA**

10:35–10:50

**Do rozpoczęcia: 08:32**

Muzyka: **TAK**

**TOP 5**

1. Believer
2. Blinding Lights
3. Titanium
4. Animals
5. Counting Stars

**10. Panel administratora**

Administrator powinien posiadać dostęp do aplikacji webowej.

Panel powinien umożliwiać m.in.:

**Utwory**

- dodawanie utworów,

- edycję,
- usuwanie,
- wyszukiwanie,
- filtrowanie.

**Użytkownicy**

- przeglądanie użytkowników,
- zarządzanie kontami,
- nadawanie ról.

**Głosowanie**

- podgląd głosów,
- ranking,
- statystyki.

**Harmonogram**

- zarządzanie przerwami,
- kalendarz,
- dni wolne,
- wyjątki.

**Odtwarzanie**

- aktualnie odtwarzany utwór,
- kolejka,
- pomijanie utworu,
- ręczna zmiana kolejności.

**11. Serwer i API**

Centralnym elementem projektu powinien być serwer aplikacji.

Serwer powinien udostępniać API umożliwiające komunikację pomiędzy:

**Aplikacją mobilną ↔ API ↔ bazą danych**

oraz

**Panelem WWW ↔ API ↔ bazą danych**

API powinno obsługiwać m.in.:

- logowanie,
- użytkowników,
- utwory,
- głosowanie,
- ranking,
- harmonogram,
- kalendarz,
- kolejkę odtwarzania,
- historię odtwarzania.

Dodatkowym atutem projektu będzie zastosowanie **WebSocketów, WebTransport itp**, pozwalających na aktualizowanie informacji w czasie rzeczywistym.

## 12. Proponowane technologie

Organizator rekomenduje wykorzystanie nowoczesnego stosu technologicznego:

**Backend**

**Node.js + NestJS + TypeScript**

**Frontend WWW**

**React + TypeScript + Next.js**

**Aplikacja mobilna**

**React Native + Expo**

**Baza danych**

**PostgreSQL**

**Komunikacja**

**REST API**

Dodatkowo:

- np. WebSocket,

- Docker,
- Git,
- GitHub,
- Swagger / OpenAPI.

Uczestnicy mogą zastosować inne technologie, pod warunkiem że umożliwiają one realizację wszystkich wymagań konkursowych.

**13. Bezpieczeństwo**

Projekt powinien uwzględniać podstawowe zasady bezpieczeństwa aplikacji.

W szczególności należy zadbać o:

- bezpieczne logowanie,
- ochronę haseł,
- autoryzację użytkowników,
- podział uprawnień,
- zabezpieczenie API,
- walidację danych,
- ochronę przed wielokrotnym głosowaniem,
- niewyświetlanie danych wrażliwych.

**14. Wymagania dotyczące projektu**

Projekt powinien być wykonany w sposób umożliwiający jego uruchomienie przez komisję konkursową.

Uczestnicy powinni przygotować:

1. kod źródłowy projektu,
2. bazę danych lub skrypt umożliwiający jej utworzenie,
3. instrukcję uruchomienia,
4. dokumentację techniczną,
5. dokumentację API,
6. prezentację projektu,

7. działającą wersję demonstracyjną.

Kod powinien „również” znajdować się w repozytorium Git.

**15. Prezentacja projektu**

Każdy zespół otrzyma określony czas na zaprezentowanie projektu przed komisją.

Podczas prezentacji należy przedstawić:

1. pomysł i sposób działania systemu,
2. architekturę rozwiązania,
3. bazę danych,
4. aplikację serwerową,
5. panel administracyjny,
6. aplikację mobilną,
7. mechanizm głosowania,
8. harmonogram przerw,
9. mechanizm tworzenia kolejki,
10. integrację ze źródłem muzyki.

Komisja może poprosić uczestników o wykonanie określonych operacji w systemie.

**16. Punktacja**

Maksymalna liczba punktów: **100 pkt**

| **Element** | **Punkty** |
|---|---|
| Funkcjonalność systemu | 20 pkt |
| Aplikacja mobilna | 15 pkt |
| Panel WWW | 10 pkt |
| Backend i REST API | 10 pkt |
| Baza danych i jej struktura | 10 pkt |
| Harmonogram i kalendarz | 10 pkt |

| Element | Punkty |
|---|---|
| System głosowania i ranking | 10 pkt |
| Kolejka odtwarzania | 5 pkt |
| Interfejs użytkownika i UX | 5 pkt |
| Dokumentacja i jakość kodu | 5 pkt |
| **Łącznie** | **100 pkt** |

**Punkty dodatkowe / wyróżnienia**

Komisja może przyznać dodatkowe wyróżnienia za szczególnie interesujące rozwiązania, takie jak:

- komunikacja WebSocket,
- powiadomienia push,
- Docker,
- automatyczne testy,
- CI/CD,
- Swagger/OpenAPI,
- rozbudowane statystyki,
- ekran „School Music” do wyświetlania w szkole,
- integracja z systemem szkolnym,
- szczególnie interesujące rozwiązania UX/UI.

## 17. Kryteria oceny

Komisja będzie zwracać szczególną uwagę na:

**Funkcjonalność**

Czy system rzeczywiście realizuje założenia zadania?

**Jakość techniczna**

Czy zastosowana architektura jest logiczna, czytelna i poprawna?

**Baza danych**

Czy struktura bazy danych jest prawidłowo zaprojektowana?

**Aplikacja mobilna**

Czy aplikacja jest wygodna i intuicyjna dla ucznia?

**Panel administratora**

Czy administrator może sprawnie zarządzać systemem?

**UX/UI**

Czy interfejs jest czytelny, nowoczesny i spójny?

**Bezpieczeństwo**

Czy uczestnicy uwzględnili podstawowe zagrożenia związane z aplikacjami internetowymi?

**Prezentacja**

Czy zespół potrafi wyjaśnić sposób działania oraz zastosowane rozwiązania?

## 18. Praca zespołowa

Projekt może być realizowany indywidualnie lub w zespołach.

W przypadku zespołu zaleca się podział obowiązków, np.:

**Backend Developer**
odpowiedzialny za serwer, API i bazę danych.

**Frontend Developer**
odpowiedzialny za panel WWW.

**Mobile Developer**
odpowiedzialny za aplikację mobilną.

**Database / DevOps**
odpowiedzialny za bazę danych, wdrożenie i środowisko uruchomieniowe.

Podział może być dowolny – wszyscy członkowie zespołu powinni jednak znać ogólne działanie całego systemu.

## 19. Ważne informacje dotyczące muzyki

Projekt powinien wykorzystywać **legalne źródła muzyki oraz oficjalne mechanizmy udostępniane przez dostawców usług**.

Uczestnicy nie mogą:

- pobierać chronionej muzyki w sposób niezgodny z regulaminem serwisu,
- omijać zabezpieczeń,
- udostępniać plików muzycznych bez odpowiednich praw,
- wykorzystywać nielegalnych źródeł.

Celem konkursu jest stworzenie **systemu informatycznego zarządzającego muzyką**, a nie stworzenie nielegalnego serwisu dystrybucji utworów.

**20. Oczekiwany efekt końcowy**

Końcowym rezultatem powinien być działający system, który pozwoli przeprowadzić następujący scenariusz:

**08:00**

Uczeń uruchamia aplikację mobilną.

Widoczna jest informacja:

🎵 Następna przerwa: 08:45
⏳ Pozostało: 45 minut

Uczeń przegląda listę 100 utworów i oddaje głos.

**08:40**

System analizuje wyniki głosowania i tworzy kolejkę:

1. Utwór A
2. Utwór B
3. Utwór C

**08:45**

Rozpoczyna się przerwa.

System automatycznie rozpoczyna odtwarzanie muzyki.

Aplikacja pokazuje:

🔴 PRZERWA TRWA

Pozostało: **09:47**

🎵 Teraz odtwarzany:
**Utwór A – Wykonawca**

**08:55**

Przerwa kończy się.

System zatrzymuje odtwarzanie i przygotowuje się do kolejnej przerwy.

**21. Główne wyzwanie konkursowe**

Nie chodzi wyłącznie o stworzenie aplikacji, która „działa”.

**Waszym zadaniem jest stworzenie kompletnego produktu informatycznego.**

Powinniście samodzielnie zdecydować:

- jak zaprojektować bazę danych,
- jak podzielić system na moduły,
- jak zabezpieczyć API,
- jak zaprojektować aplikację mobilną,
- jak stworzyć algorytm tworzenia kolejki,
- jak rozwiązać konflikt między długością utworów a czasem trwania przerwy,
- jak obsłużyć dni wolne i wyjątki w kalendarzu,
- jak zapewnić aktualizację danych w czasie rzeczywistym,
- jak zaprojektować interfejs dla ucznia i administratora.

**Najważniejsze jest stworzenie rozwiązania, które mogłoby zostać rzeczywiście wykorzystane w szkole.**

**22. Hasło projektu**

🎵 **SCHOOL MUSIC**

**„Ty głosujesz. Szkoła gra.”**